

Campus Events Manager

MongoDB-Based Web Application

Design, implementation and analytical reporting

Executive Summary

The Campus Events Manager is a database-driven web application for managing university users, events and registrations. It demonstrates practical NoSQL development through MongoDB document modelling, schema validation, indexing, CRUD operations, embedded documents, references, array updates and multi-stage aggregation pipelines.

The solution provides five main functional areas:

1. A dashboard summarising campus activity.
2. Event search, filtering, creation, editing and deletion.
3. Event details and registration management.
4. User management and participation history.
5. Six analytical reports calculated directly in MongoDB.

Technology Stack

FastAPI

PyMongo

MongoDB

Jinja2

Bootstrap

Pytest

Project Scope

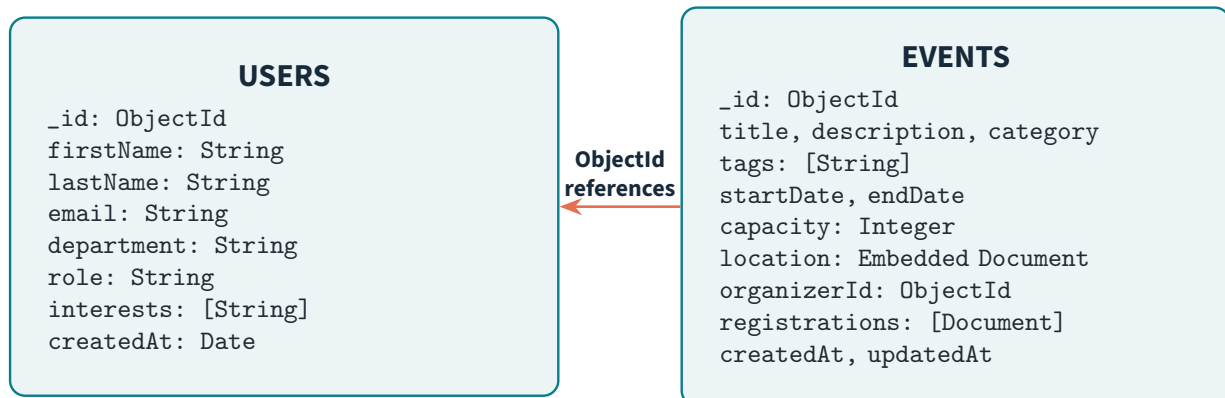
The implementation focuses on MongoDB and application development. Authentication, payments, email verification, real-time notifications, Docker, complex security and cloud deployment are outside the required scope.

Name: Sai Kishan Kumar
Institution: Aivancity School for Technology, Business & Society
Course: NoSQL Development Project
Submission: 12 September 2026

1 NoSQL Document Model

The application uses the MongoDB database `campus_events`. Its operational data is stored in two collections: `users` and `events`.

1.1 Collection Structure



1.2 Embedding and Referencing Decisions

Embedded location

A location belongs to one event and is normally read together with that event. Embedding the building, room and address avoids an unnecessary collection and permits the complete event to be retrieved in a single operation.

Embedded registrations

Registrations are closely associated with an event. Embedding them supports direct capacity calculations, duplicate checks and atomic updates using `$addToSet` and `$pull`. Each registration stores a user reference, registration timestamp and status.

Referenced users

Organizers and participants are represented by user `ObjectId` references. Users have an independent lifecycle and may participate in several events. Referencing avoids duplicating names, emails, roles and departments.

1.3 Validation and Indexing

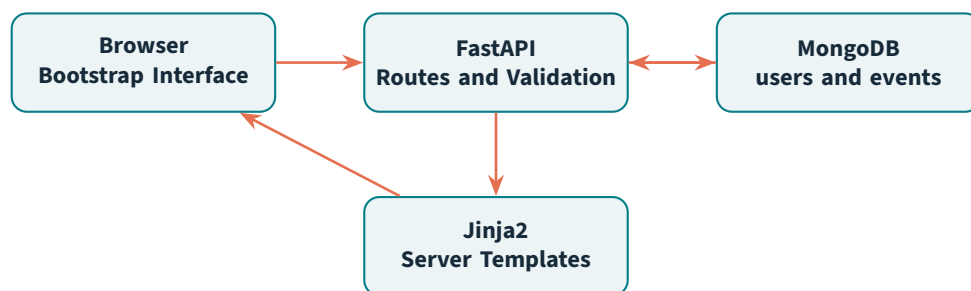
MongoDB JSON Schema validators enforce required fields, BSON data types, accepted roles, registration statuses and positive capacities. The database includes a unique index on user email and indexes on event date, category, tags and organizer.

Result

The model combines the read efficiency of embedded data with the consistency and reuse provided by references.

2 Architecture and Core Application

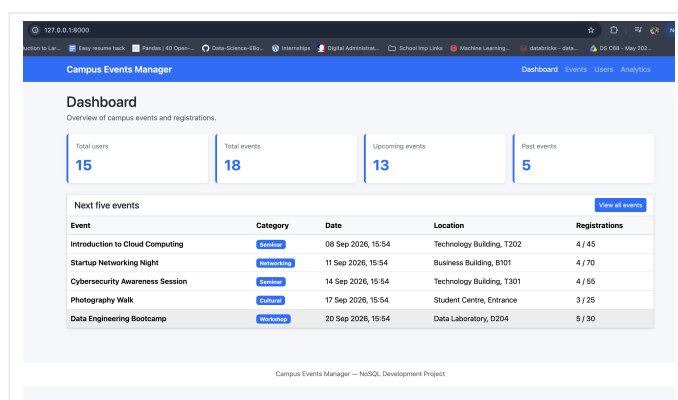
2.1 Application Architecture



FastAPI handles routing, form processing and validation. PyMongo executes direct operations against MongoDB. Jinja2 produces server-rendered pages, and Bootstrap provides a responsive interface.

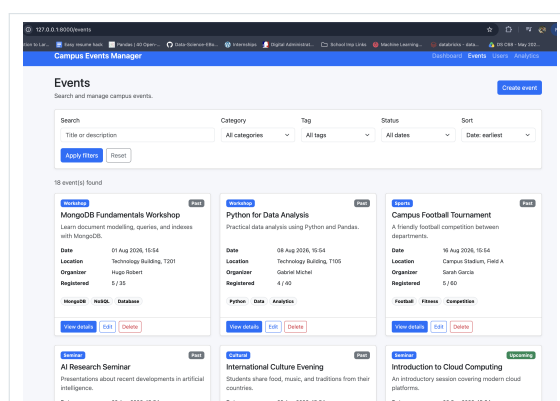
Configuration is loaded from environment variables. Separate scripts initialize collection validators and indexes and generate the reproducible demonstration dataset.

2.2 Dashboard



The dashboard displays total users, total events, upcoming events, past events and the next five scheduled events. All figures are calculated directly from MongoDB.

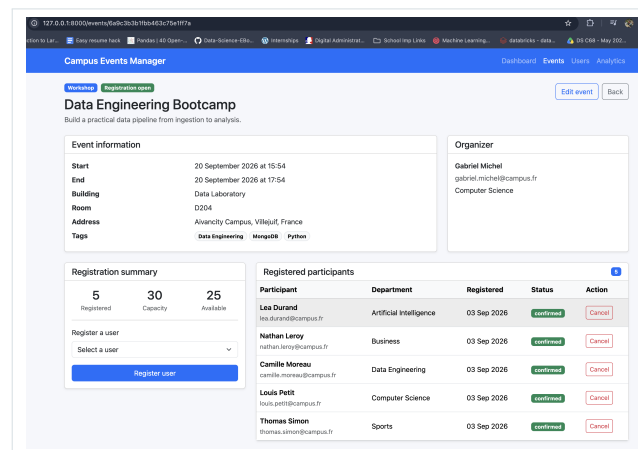
2.3 Event Discovery and Management



The Events page provides title and description search, category and tag filters, event-status filters, capacity filtering and sorting. Users can create, inspect, edit and delete event records through the interface.

3 Registration and User Management

3.1 Event Details and Registrations



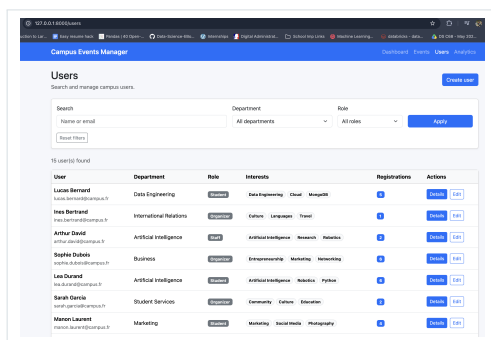
The event-details page combines embedded event information with referenced organizer and participant records. It presents the location, schedule, tags, capacity, remaining places and registered participants.

Registration uses an atomic MongoDB update with:

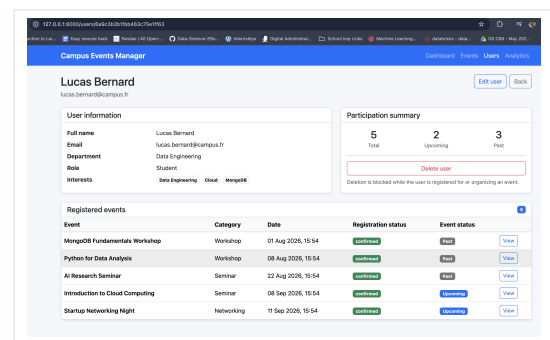
- A filter preventing duplicate user registrations.
- An expression comparing registration count with capacity.
- `$addToSet` to insert an embedded registration.
- `$set` to maintain the update timestamp.

Cancellation uses `$pull` to remove the selected registration.

3.2 Users and Participation History



Searchable user list and filters



Participation history

The Users page supports name and email search, department and role filters, interests and registration counts. Creation and editing are validated on the server. A unique MongoDB index prevents duplicate email addresses.

The user-details page displays every registered event, registration status and separate upcoming and past totals. Users without registrations retain a valid details page. Deletion is blocked when a user is referenced as an organizer or participant, preventing orphaned references.

Consistency Rules

Duplicate registrations are rejected, event capacity is enforced, user emails remain unique and referenced users cannot be deleted inconsistently.

4 MongoDB Analytics

The Analytics page implements the six mandatory analyses:

1. Confirmed registrations by category.
2. Top five events by confirmed registrations.
3. Users with no event registration.
4. Events above average occupancy.
5. Most-used event tags.
6. Events and registrations grouped by month.

Analytics
Compare event insights calculated using MongoDB aggregation pipelines.

Categories analyzed: 8 | Users without registrations: 0 | Average occupancy: 7.69%

A. Registrations by category
Categories ordered by confirmed registrations.

Category	Number of events	Confirmed registrations
Workshop	4	16
Seminar	4	16
Cultural	3	9
Sports	2	8
Career	1	4
Competition	1	4
Networking	2	4
Technology	1	4

B. Top five most popular events
Ordered by confirmed registrations.

Rank	Event	Category	Capacity	Confirmed	Occupancy
1	Campus Football Tournament	Sports	60	5	8.33%
2	Data Analytics Workshop	Workshop	30	5	16.67%
3	Introduction to Data Science	Cultural	40	5	12.50%
4	MongoDB Fundamentals Workshop	Workshop	30	5	16.67%

D. Events above average occupancy
Overall average: 7.69%

Event	Category	Capacity	Confirmed	Occupancy
Data Analytics Workshop	Workshop	30	5	16.67%
MongoDB Fundamentals Workshop	Workshop	30	5	16.67%
Introduction to Data Science	Cultural	40	5	12.50%
Python for Data Analysis	Workshop	40	4	10.00%
Web Development Workshop	Competition	40	4	10.00%
Campus Football Tournament	Sports	60	5	8.33%
Introduction to Data Science	Seminar	40	4	8.00%
Campus Football Tournament	Sports	60	5	8.33%
AI Research Seminar	Seminar	50	4	8.00%

E. Most-used tags
Tags ordered from most frequent to least frequent.

Rank	Tag	Number of events
1	Workshop	5
2	Seminar	3
3	Cultural	3
4	Sports	2
5	Competition	2
6	Networking	2
7	Technology	2
8	Career	2

4.1 Pipeline 1: Events Above Average Occupancy

The pipeline filters embedded registrations to retain confirmed records and calculates the confirmed count for each event. Occupancy is calculated as:

$$\text{Occupancy percentage} = \frac{\text{Confirmed registrations}}{\text{Capacity}} \times 100$$

A grouping stage calculates the overall average with `$avg` and retains event results with `$push`. After unwinding the results, `$match` and `$expr` retain only events whose occupancy exceeds the overall average.

```
$project → $project → $group → $unwind → $match → $sort
```

4.2 Pipeline 2: Users Without Registrations

Starting from the users collection, `$lookup` joins each user to events through `registrations.userId`. A subsequent `$match` retains users whose joined event array is empty. The final stages project the required user information and sort the result alphabetically.

```
$lookup → $match → $project → $sort
```

The analytics collectively demonstrate `$match`, `$project`, `$group`, `$sort`, `$unwind`, `$lookup`, `$sum`, `$avg`, array processing and calculated fields.

5 Challenges, Limitations and Conclusion

5.1 Development Challenges

Timezone consistency

MongoDB stores dates in UTC, while PyMongo initially returned timezone-naive Python values. Comparing those values with timezone-aware timestamps caused an application error. Configuring the client with `tz_aware=True` established consistent UTC handling throughout the application.

Combining embedded and referenced data

Registrations are embedded inside events, while participant details remain in the users collection. The application extracts registration user identifiers, retrieves the referenced users and combines both sources for event details and participation-history views.

Atomic registration rules

Application-level checks alone cannot guarantee correctness when multiple requests occur close together. Duplicate and capacity conditions were therefore included in the MongoDB update filter before adding the registration.

Referential consistency

Deleting a referenced user could leave invalid organizer or participant references. The deletion workflow checks both reference paths and blocks deletion whenever a dependency exists.

5.2 Current Limitations

- The application does not include authentication or role-based authorization.
- Dates use one application timezone rather than a user-selectable timezone.
- Analytics are displayed through KPI cards and tables rather than interactive charts.
- The solution is intended for local execution rather than public cloud hosting.
- Registration statuses do not include a separate approval workflow.

5.3 Potential Improvements

A production version could add authentication, authorization, pagination, interactive charts, audit logs, user-selectable timezones, email reminders and cloud deployment. These improvements are outside the current academic scope.

5.4 Conclusion

The Campus Events Manager satisfies the functional and technical objectives of the NoSQL Development Project. It provides a reproducible MongoDB database, validated document structures, indexes, CRUD operations, embedded and referenced relationships, registration management, user participation history and six analytical pipelines.

The project demonstrates how MongoDB can support operational workflows and analytical queries within one coherent application. FastAPI, PyMongo and server-rendered templates provide a transparent architecture in which the required MongoDB operations remain clearly identifiable and reproducible.

Completed Deliverables

- Complete application source code.
- MongoDB initialization and seed scripts.
- Environment-based configuration.
- Automated tests and error handling.
- Reproducible installation instructions.
- Final six-page technical report.

END OF REPORT